In [7]:
```python
# Python imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import yfinance as yf
import sqlite3
from pathlib import Path

# Visualization settings
plt.style.use('seaborn-v0_8-darkgrid')
```

In [8]:
```python
# ------------------------------
# Path setup
# ------------------------------
db_path = Path("../stockbroking/portfolio_project/data/portfolio.db
db_path.parent.mkdir(parents=True, exist_ok=True)

# ------------------------------
# Create fresh DB and tables
# ------------------------------
conn = sqlite3.connect(db_path)

conn.execute("DROP TABLE IF EXISTS asset_prices")
conn.commit()


conn.execute('''
CREATE TABLE asset_prices (
    asset_name TEXT,
    date DATE,
    price REAL,
    dividend REAL DEFAULT 0,
    PRIMARY KEY (asset_name, date)
)
''')

conn.execute("DROP TABLE IF EXISTS portfolio_weights")
conn.commit()

conn.execute('''
CREATE TABLE portfolio_weights (
    asset_name TEXT PRIMARY KEY,
    weight REAL
)
''')

conn.commit()
conn.close()

# ------------------------------
# Download & Insert
# ------------------------------
assets = ["AAPL", "JPM", "BP.L", "ULVR.L", "CSP1.L"]
```

```
start_date = "2018-01-01"
end_date = "2026-01-20"

conn = sqlite3.connect(db_path)



for asset in assets:
    print(f"Downloading {asset} data...")
    try:
        df = yf.download(asset, start=start_date, end=end_date)
        if df.empty:
            print(f"No data for {asset}, skipping...")
            continue

        df.reset_index(inplace=True)
        df.rename(columns={'Date':'date', 'Close':'price'}, inplace
        df['asset_name'] = asset
        df_to_sql = df[['asset_name','date','price']]

        # Flatten columns in case of tuple column names
        df_to_sql.columns = [c if isinstance(c, str) else c[0] for

        # Insert into SQLite
        df_to_sql.to_sql('asset_prices', conn, if_exists='append',
        print(f"{asset} inserted successfully.")

    except Exception as e:
        print(f"Error downloading {asset}: {e}")


conn.close()
```

```
[**********************100%**********************]  1 of 1 completed
[**********************100%**********************]  1 of 1 completed
[**********************100%**********************]  1 of 1 completed
Downloading AAPL data...
AAPL inserted successfully.
Downloading JPM data...
JPM inserted successfully.
Downloading BP.L data...
BP.L inserted successfully.
Downloading ULVR.L data...
[**********************100%**********************]  1 of 1 completed
[**********************100%**********************]  1 of 1 completed
ULVR.L inserted successfully.
Downloading CSP1.L data...
CSP1.L inserted successfully.
```

In [9]:

```
# ------------------------------
# Connect to DB & Load Prices
# ------------------------------
db_path = "../stockbroking/portfolio_project/data/portfolio.db"
conn = sqlite3.connect(db_path)

# Pull all asset prices
```

```
prices_df = pd.read_sql_query("SELECT * FROM asset_prices", conn, p
conn.close()

# Pivot to have assets as columns
prices_pivot = prices_df.pivot(index='date', columns='asset_name',
prices_pivot.sort_index(inplace=True)

print("Price Data Sample:")
print(prices_pivot.head())

# ------------------------------
# Calculate Daily Returns
# ------------------------------
returns = prices_pivot.pct_change().dropna()
print("\nDaily Returns Sample:")
print(returns.head())

# ------------------------------
# Define Portfolio Weights
# ------------------------------
# equal weights
weights = pd.Series({
    "AAPL": 0.2,
    "JPM": 0.2,
    "BP.L": 0.2,
    "ULVR.L": 0.2,
    "CSP1.L": 0.2
})

# save weights to DB
conn = sqlite3.connect(db_path)
weights_df = weights.reset_index()
weights_df.columns = ['asset_name', 'weight']
weights_df.to_sql('portfolio_weights', conn, if_exists='replace', i
conn.close()

# ------------------------------
# Calculate Portfolio Daily Returns
# ------------------------------
portfolio_returns = returns.dot(weights)
portfolio_returns = portfolio_returns.to_frame(name='portfolio_retu

print("\nPortfolio Returns Sample:")
print(portfolio_returns.head())

# ------------------------------
# Visualization
# ------------------------------

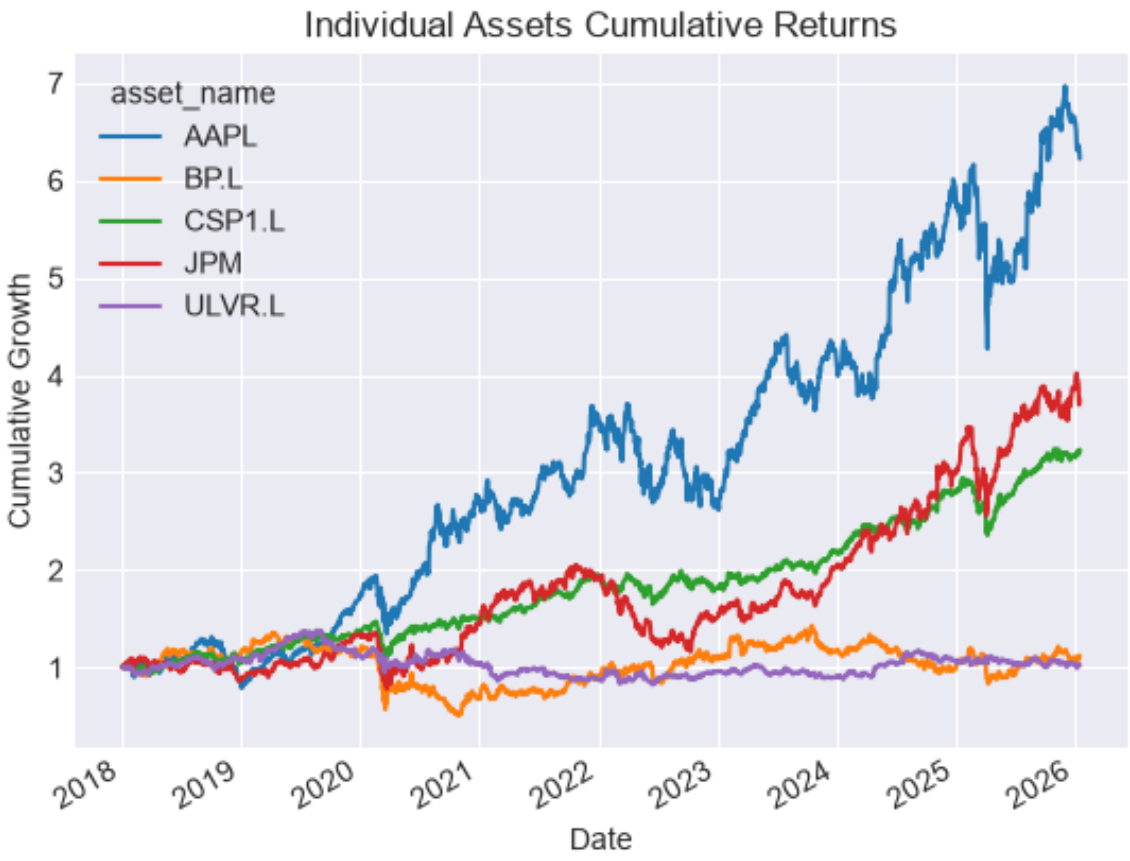
# Individual assets cumulative returns
plt.figure(figsize=(12,6))
(1 + returns).cumprod().plot()
plt.title("Individual Assets Cumulative Returns")
plt.ylabel("Cumulative Growth")
plt.xlabel("Date")
plt.grid(True)
```

```
plt.show()
```

```
Price Data Sample:
asset_name       AAPL        BP.L   CSP1.L        JPM       ULVR.L
date
2018-01-02  40.232372  515.035950  19060.0  85.091507  4312.446777
2018-01-03  40.225380  521.703979  19275.5  85.178238  4326.839355
2018-01-04  40.412228  527.476196  19351.5  86.398453  4340.165527
2018-01-05  40.872322  527.078064  19399.5  85.843796  4363.087402
2018-01-08  40.720509  524.888794  19451.5  85.970566  4362.020020

Daily Returns Sample:
asset_name      AAPL      BP.L    CSP1.L       JPM    ULVR.L
date
2018-01-03 -0.000174  0.012947  0.011306  0.001019  0.003337
2018-01-04  0.004645  0.011064  0.003943  0.014325  0.003080
2018-01-05  0.011385 -0.000755  0.002480 -0.006420  0.005281
2018-01-08 -0.003714 -0.004154  0.002680  0.001477 -0.000245
2018-01-09 -0.000115  0.001137  0.009074  0.005069 -0.001833

Portfolio Returns Sample:
            portfolio_return
date
2018-01-03          0.005687
2018-01-04          0.007411
2018-01-05          0.002394
2018-01-08         -0.000791
2018-01-09          0.002667
<Figure size 1200x600 with 0 Axes>
```



<p>This shows shows how the portfolio grows overtime if all returns are reinvested. AAPL showed most cumulative growth while the more noncyclical

assets like BP.L and ULVR.L showed the least volatility.</p>

In [10]:
```
# ------------------------------
# Annualized Portfolio Metrics
# ------------------------------
trading_days = 252  # typical trading days in a year

# Annualized return
annualized_return = portfolio_returns['portfolio_return'].mean() *

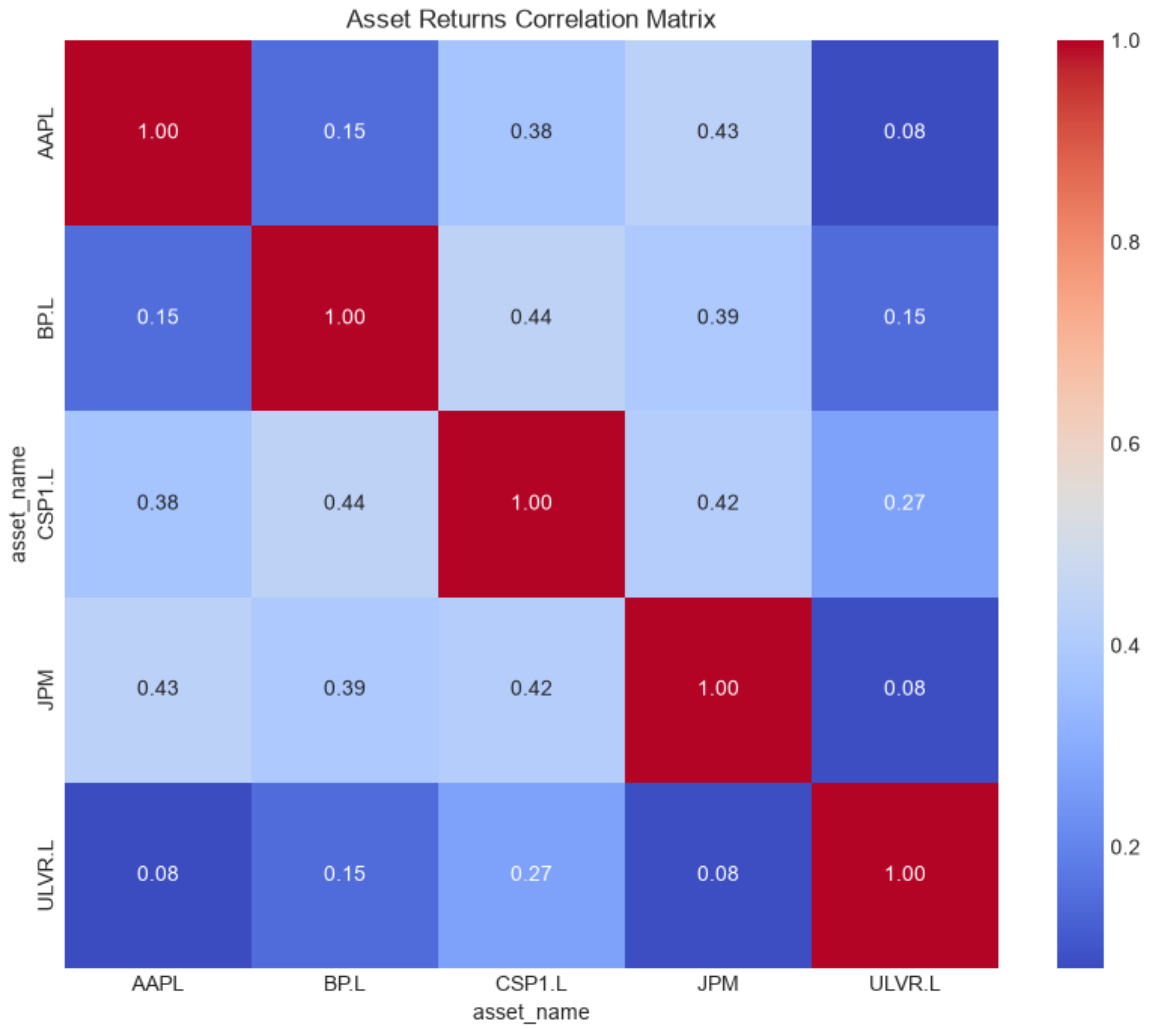
# Annualized volatility
annualized_vol = portfolio_returns['portfolio_return'].std() * np.s

# Sharpe ratio (assume risk-free = 0)
sharpe_ratio = annualized_return / annualized_vol

print(f"Annualized Portfolio Return: {annualized_return:.2%}")
print(f"Annualized Portfolio Volatility: {annualized_vol:.2%}")
print(f"Sharpe Ratio: {sharpe_ratio:.2f}")

# ------------------------------
# Correlation Matrix
# ------------------------------
corr_matrix = returns.corr()
plt.figure(figsize=(10,8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Asset Returns Correlation Matrix")
plt.show()
```

```
Annualized Portfolio Return: 15.33%
Annualized Portfolio Volatility: 16.86%
Sharpe Ratio: 0.91
```



This shows how the assets move relative to each other.Low or negative correlations reduce the risk of the portfolio - these are indicated by the blue to dark blue colour which dominates majority of the portfolio showing our risk is fairly moderate currently.

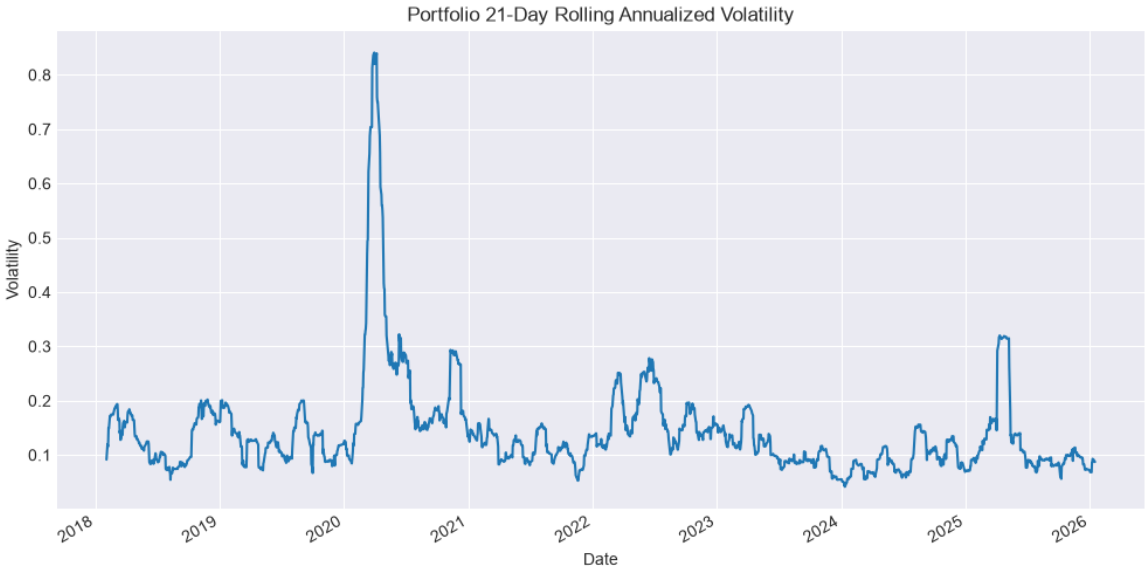
In [11]:

```
# ------------------------------
# Value at Risk (VaR) – 95% confidence
# ------------------------------
# Parametric VaR (Normal)
portfolio_std = portfolio_returns['portfolio_return'].std()
VaR_95 = -1.65 * portfolio_std  # 1.65 ~ 95% quantile of normal
print(f"Portfolio 1-day VaR at 95% confidence: {VaR_95:.2%}")

# ------------------------------
# Rolling Metrics
# ------------------------------
rolling_vol = portfolio_returns['portfolio_return'].rolling(21).std

plt.figure(figsize=(12,6))
rolling_vol.plot()
plt.title("Portfolio 21-Day Rolling Annualized Volatility")
plt.ylabel("Volatility")
plt.xlabel("Date")
plt.grid(True)
plt.show()
```

```
Portfolio 1-day VaR at 95% confidence: -1.75%
```



This highlights the short-term fluctuations in portfolio risk. Spikes tend to align with market events that impact financial and tech assets (i.e. AAPL and JPM). Despite these spikes, the overall volatility remains fairly moderate.

In [12]:

```
# ------------------------------
# Step 1: Portfolio Covariance Matrix
# ------------------------------
cov_matrix = returns.cov() * trading_days  # annualized covariance

# Portfolio weights as array
w = weights.values.reshape(-1,1)

# Portfolio variance
portfolio_var = float(w.T @ cov_matrix.values @ w)
portfolio_vol = np.sqrt(portfolio_var)
print(f"Portfolio annualized volatility (check): {portfolio_vol:.2%

# ------------------------------
# Step 2: Marginal Contribution to Risk
# ------------------------------
# Marginal contribution = cov_matrix * weights / portfolio_vol
marginal_contrib = cov_matrix.values @ w / portfolio_vol

# Risk contribution (percentage of total portfolio risk)
risk_contrib_pct = (marginal_contrib * w) / portfolio_vol
risk_contrib_pct = risk_contrib_pct.flatten()

risk_df = pd.DataFrame({
    'Asset': weights.index,
    'Weight': weights.values,
    'Risk Contribution %': risk_contrib_pct * 100
})

print("\nAsset Risk Contribution to Portfolio:")
print(risk_df)

# ------------------------------
```

```
# Step 3: Plot Portfolio vs Individual Asset Cumulative Returns
# ------------------------------
plt.figure(figsize=(12,6))
(1 + returns).cumprod().plot(alpha=0.7)
(1 + portfolio_returns).cumprod().plot(color='black', linewidth=2,
plt.title("Cumulative Returns: Portfolio vs Individual Assets")
plt.ylabel("Cumulative Growth")
plt.xlabel("Date")
plt.legend()
plt.grid(True)
plt.show()
```

```
---------------------------------------------------------------------------
-------
TypeError                                 Traceback (most recent cal
l last)
Cell In[12], line 10
      6 # Portfolio weights as array
      7 w = weights.values.reshape(-1,1)
      8
      9 # Portfolio variance
---> 10 portfolio_var = float(w.T @ cov_matrix.values @ w)
     11 portfolio_vol = np.sqrt(portfolio_var)
     12 print(f"Portfolio annualized volatility (check): {portfolio_
vol:.2%}")
     13

TypeError: only 0-dimensional arrays can be converted to Python scal
ars
```

This chart shows how the portfolio grows relative to its constituent assets. AAPL had the greatest overall growth and the rest of the portfolio also showed either smooth or good returns over the period. Overall the portfolio shows very good growth and returns.

In [ ]:
```
# ------------------------------
# Step 4: Pie Chart of Risk Contribution
# ------------------------------
plt.figure(figsize=(8,8))
plt.pie(risk_df['Risk Contribution %'], labels=risk_df['Asset'], au
plt.title("Portfolio Risk Contribution by Asset")
plt.show()
```

This shows how each assets contributes to the overall portfolio volatility. JPM and ULVR.L are the largest contributors but are followed closely by AAPL. These three assets represent the main drivers in the portfolio and help in assessing whether the allocation aligns with the risk profile.

In [ ]:
```
# ------------------------------
# Portfolio Insights & Summary
# ------------------------------

import matplotlib.pyplot as plt
import numpy as np
```

```
trading_days = 252

# Annualized metrics
annualized_return = portfolio_returns['portfolio_return'].mean() *
annualized_vol = portfolio_returns['portfolio_return'].std() * np.s
sharpe_ratio = annualized_return / annualized_vol
VaR_95 = -1.65 * portfolio_returns['portfolio_return'].std()  # 1-d

print("----- Portfolio Summary -----")
print(f"Annualized Return: {annualized_return:.2%}")
print(f"Annualized Volatility: {annualized_vol:.2%}")
print(f"Sharpe Ratio: {sharpe_ratio:.2f}")
print(f"1-day 95% Portfolio VaR: {VaR_95:.2%}")

# Cumulative portfolio returns
portfolio_cum = (1 + portfolio_returns['portfolio_return']).cumprod
plt.figure(figsize=(10,5))
plt.plot(portfolio_cum, label='Portfolio')
plt.title("Portfolio Cumulative Returns")
plt.xlabel("Date")
plt.ylabel("Cumulative Return")
plt.legend()
plt.grid(True)
plt.show()

# Individual asset contribution to risk (volatility)
asset_vols = returns.std() * np.sqrt(trading_days)
risk_contrib = asset_vols * weights
risk_contrib_pct = risk_contrib / risk_contrib.sum() * 100

plt.figure(figsize=(8,5))
plt.bar(risk_contrib_pct.index, risk_contrib_pct.values, color='sky
plt.title("Risk Contribution by Asset (%)")
plt.ylabel("Contribution to Total Portfolio Risk")
plt.grid(True, axis='y')
plt.show()


# ------------------------------
# Rolling 30-day Volatility
# ------------------------------
rolling_window = 30  # trading days (~1.5 months)

rolling_vol = returns.rolling(window=rolling_window).std() * np.sqr
plt.figure(figsize=(12,6))
rolling_vol.plot()
plt.title(f"Rolling {rolling_window}-Day Annualized Volatility")
plt.xlabel("Date")
plt.ylabel("Volatility")
plt.grid(True)
plt.show()
```

<h1 id="Portfolio-Insights-&-Summary">Portfolio Insights & Summary</h1>

<h2 id="Annualized-Metrics">Annualized Metrics</h2>

```
**Annualized Return:** 13.44%
Expected yearly return if the portfolio's average
daily return continued over a full trading year.

**Annualized Volatility:** 16.55%
Measures the fluctuation of portfolio returns. Higher
volatility indicates higher risk.

**Sharpe Ratio:** 0.81
Risk-adjusted return metric. Shows how much return is
earned per unit of risk.

**1-day 95% Value at Risk (VaR):** -1.72%
Inidcates the maximum expected loss over a single day
with 95% confidence.
There's a 5% chance of losing more than 1.72% in one
day.
```

<h2 id="Portfolio-Cumulative-Returns">Portfolio Cumulative Returns</h2> The cumulative returns chart shows how much $1 invested at the start of the time period would have grown over time. It shows the compounding effects and periods where the portfolio outperformed or underperformed individual assets. <h2 id="Risk-Contribution-by-Asset">Risk Contribution by Asset</h2>

```
**AAPL:** 24%
**BP.L:** 25%
**CSP1.L:** 13%
**JPM:** 23%
**ULVR.L:** 15%
```

These figures highlight which assets dominate the portfolio risk. Larger weights and/or higher volatility assets contribute more. It helps to understand where risk is concentrated so rebalancing decisions can be made. <h2 id="Rolling-30-day-Annualized-Volatility">Rolling 30-day Annualized Volatility</h2> This highlights how the portfolio risk fluctuates overtime. Tech and financial stocks (AAPL, JPM) display higher spikes comparent to the other assets illustrating their higher sensitivity to market events. <h2 id="Diversification-Observations">Diversification Observations</h2> The portfolio equally weights 5 assets, which balances the risk contributions roughly evenly. Changing volatilities can cause risk contributions to change overtime. Good diversification will reduce overall risk and maintain return potential.

In [ ]:
```
# ------------------------------
# Step 6: Factor Analysis Automation
# ------------------------------
market_ticker = "^FTSE"
market_df = yf.download(market_ticker, start=start_date, end=end_da
```

```
market_returns = market_df['Close'].squeeze().pct_change().dropna()
market_returns = market_returns.rename("market_return")

# Align portfolio & market returns
aligned_df = pd.concat([portfolio_returns, market_returns], axis=1)

# Factor 1: Beta vs Market
cov_matrix = np.cov(aligned_df['portfolio_return'], aligned_df['mar
beta = cov_matrix[0,1] / cov_matrix[1,1]
alpha = aligned_df['portfolio_return'].mean() * trading_days - beta

print("----- Factor Analysis -----")
print(f"Beta vs {market_ticker}: {beta:.4f}")
print(f"Alpha vs {market_ticker} (annualized): {alpha:.2%}")

# Factor 2: Volatility / Risk
portfolio_vol = aligned_df['portfolio_return'].std() * np.sqrt(trad
print(f"Annualized Portfolio Volatility: {portfolio_vol:.2%}")

# Factor 3: Rolling Correlation with Market
rolling_corr = aligned_df['portfolio_return'].rolling(21).corr(alig
plt.figure(figsize=(12,6))
plt.plot(rolling_corr, label='21-day Rolling Correlation')
plt.title("Portfolio vs Market Rolling Correlation")
plt.xlabel("Date")
plt.ylabel("Correlation")
plt.legend()
plt.show()
```

This factor analysis compares your portfolio to the FTSE 100. Beta (0.831): The portfolio is slightly less sensitive to market movements than the FTSE 100. In other words, it tends to fluctuate less than the market overall. Alpha (8.85% annualized): After accounting for market movements, the portfolio is generating a strong positive excess return. This means it has historically outperformed what would be expected given its Beta. Annualized Volatility (16.66%): The overall risk level of the portfolio, as measured by standard deviation of returns, is moderate. Combined with the Beta < 1, this suggests the portfolio achieves decent returns with less market-driven risk. The 21-day rolling correlation chart visualizes how the portfolio's daily returns move relative to the FTSE over time. Periods of lower correlation indicate times when the portfolio behaved more independently from the market, providing potential diversification benefits.

In [ ]: